Note de séance SQL n°3

Les jointures : INNER JOIN, LEFT JOIN | 14 octobre 2026

1. L'idée de la séance

Dans une vraie base, les données sont réparties dans plusieurs tables. Une **jointure** relie deux tables grâce à une colonne commune : ici, `clients.id` et `commandes.client_id`.

2. La table commandes

```
CREATE TABLE commandes (
    id INT,
    client_id INT,
    produit VARCHAR(50),
    montant INT
);

INSERT INTO commandes VALUES (1, 1, 'Telephone', 250);
INSERT INTO commandes VALUES (2, 1, 'Casque', 40);
INSERT INTO commandes VALUES (3, 2, 'Ordinateur', 800);
INSERT INTO commandes VALUES (4, 3, 'Souris', 15);
INSERT INTO commandes VALUES (5, 3, 'Clavier', 30);
INSERT INTO commandes VALUES (6, 5, 'Tablette', 300);
INSERT INTO commandes VALUES (7, 9, 'Montre', 120);
```

Cas volontaires : les clients 4, 6, 7 et 8 n'ont aucune commande, et la commande 7 appartient au client 9, qui n'existe pas.

3. Les types de jointures

Jointure	Ce qu'elle garde	Résultat ici
INNER JOIN	Seulement les lignes avec correspondance des deux côtés	6 lignes
LEFT JOIN	Toutes les lignes de la table de gauche, NULL si pas de correspondance	10 lignes

Un alias de table raccourcit l'écriture : `FROM clients c INNER JOIN commandes o ON c.id = o.client_id`.

4. Exercices et corrections

```
-- INNER JOIN : prénom, ville et produit de chaque commande (6 lignes)
SELECT c.prenom, c.ville, o.produit
FROM commandes o
INNER JOIN clients c ON o.client_id = c.id;

-- LEFT JOIN : tous les clients, même sans commande (10 lignes)
SELECT c.prenom, o.*
FROM clients c
LEFT JOIN commandes o ON c.id = o.client_id;

-- Clients sans aucune commande : Patrick, David, Sarah, Eric
SELECT c.prenom, o.*
FROM clients c
```

```
LEFT JOIN commandes o ON c.id = o.client_id
WHERE o.id IS NULL;

-- Dépense par client : Awa 290 / Jean 800 / Marie 45 / Grace 300
SELECT c.id, c.prenom, SUM(o.montant) AS depense
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
GROUP BY c.id, c.prenom;
```

5. Mini-défis et corrections

```
-- Total par ville : Kinshasa 635 / Lubumbashi 800
SELECT c.ville, SUM(o.montant) AS total
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
GROUP BY c.ville;

-- Le client qui a le plus dépensé : Jean (800)
SELECT c.id, c.prenom, SUM(o.montant) AS depense
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
GROUP BY c.id, c.prenom
ORDER BY depense DESC
LIMIT 1;

-- Tous les clients avec leur dépense, 0 si aucune commande
SELECT c.id, c.prenom, COALESCE(SUM(o.montant), 0) AS depense
FROM clients c
LEFT JOIN commandes o ON c.id = o.client_id
GROUP BY c.id, c.prenom;

-- Commandes sans client (commande 7, client 9 inconnu)
SELECT c.*, o.*
FROM commandes o
LEFT JOIN clients c ON c.id = o.client_id
WHERE c.id IS NULL;
```

6. À retenir

• **INNER JOIN** : seulement les correspondances. **LEFT JOIN** : tout le côté gauche, avec NULL quand il n'y a rien en face.

• Trouver les lignes sans correspondance : `LEFT JOIN ... WHERE autre_table.id IS NULL`. Un classique d'entretien.

• Le choix INNER ou LEFT dépend de la question : seulement ceux qui ont commandé, ou tout le monde ?

• `COALESCE(valeur, 0)` remplace NULL par 0 et fonctionne sur tous les systèmes (IFNULL est propre à MySQL).

• Regrouper par identifiant (`GROUP BY c.id, c.prenom`) évite de fusionner deux clients qui auraient le même prénom.

• Les jointures permettent de détecter des données incohérentes, comme la commande orpheline : une vraie tâche de data analyste.

7. Prochaine séance

Les sous-requêtes et les CTE (WITH) : construire des requêtes en plusieurs étapes.